🎯 DESARROLLO WEB EN ENTORNO CLIENTE · PROYECTOS DE AULA

⏱️ PRÁCTICAS GUIADAS: TEMPORIZADORES

Cuaderno de aula con dos proyectos interactivos completos sobre plantillas HTML/CSS preconstruidas: desarrollo paso a paso, código de solución oficial, arquitectura de control de estados y verificación cruzada de resultados.

🚀 Práctica 1: Lanzamiento de Cohete

Panel de control de misión espacial con cuenta atrás de 10 a 0, abortado de emergencia en seco y restauración diferida del sistema a los 3 segundos.

`setInterval() + clearInterval() + setTimeout()`

🔋 Práctica 2: Simulador de Batería

Indicador de carga interactivo del 0% al 100% con botón de desconexión / pausa, alerta temporal diferida y cancelación segura ante reinicios.

`setInterval() + clearInterval() + setTimeout() + clearTimeout()`

👨‍🏫 **Ramón Abramo**
Profesor especialista con más de 25 años de experiencia docente

📑 Índice General de las Prácticas (Haz clic para saltar)

Prácticas Guiadas

📋 Instrucciones Generales para el Alumno	Pág 2	🚀 Práctica 1: Enunciado y Misión Espacial	Pág 3
🧱 Práctica 1: Código Base HTML y CSS	Pág 3	🧭 Práctica 1: Guía de Desarrollo Paso a Paso	Pág 4
🖥️ Práctica 1: Solución Oficial Comentada	Pág 4	🔋 Práctica 2: Enunciado del Simulador de Batería	Pág 5
🧱 Práctica 2: Código Base HTML y CSS	Pág 5	🧭 Práctica 2: Guía de Desarrollo Paso a Paso	Pág 6
🖥️ Práctica 2: Solución Oficial Comentada	Pág 6	📊 Gran Tabla de Verificación y Autoevaluación	Pág 6

📋 Instrucciones Generales para el Alumnado

💡 Metodología de Trabajo en el Aula

En ambas prácticas se entrega todo el código HTML y CSS finalizado y estilizado. **Tu tarea consiste exclusivamente en programar la lógica en el archivo** `app.js` siguiendo los pasos ordenados y aplicando lo aprendido en clase. **No modifiques los atributos** `id` **ni las clases CSS** del documento base para no quebrar las capturas del DOM.

🛠️ CONFIGURACIÓN DEL ENTORNO DE DESARROLLO

1. Abre la carpeta de la práctica en Visual Studio Code u otro editor web profesional.
2. Inicia el servidor local (extensión *Live Server*) o haz doble clic sobre `index.html`.
3. Pulsa `F12` en el navegador y abre la pestaña **Consola**. Allí se reportan al instante los errores de sintaxis y excepciones.

⚠️ Regla de Oro Contra Intervalos Descontrolados

Nunca lances un `setInterval` sin haber guardado previamente su identificador en una variable (`idIntervalo = setInterval( ... )`) y sin validar si ya existía una ejecución activa en marcha. De lo contrario, cada pulsación del usuario multiplicará la velocidad de ejecución exponencialmente.

🚀 Práctica 1: Panel de Lanzamiento de Cohete con Abortado

🎯 DATOS TÉCNICOS DE LA PRÁCTICA

Funciones a utilizar: `setInterval()`, `clearInterval()` y `setTimeout()` · **Carpeta alumno:** `practicas/practica_01_cohete/`

📖 Enunciado de la Misión Espacial

Debes programar el panel de control de lanzamiento de un cohete espacial con las siguientes especificaciones:

1. Al pulsar el botón **«Iniciar cuenta atrás»**, el marcador descontará segundos desde **10 hasta 0** (restando 1 segundo en cada ciclo de 1000 ms).
2. Mientras la cuenta atrás esté en marcha, el botón de iniciar quedará deshabilitado (`disabled = true`) y se habilitará el botón **«Abortar lanzamiento»** (`disabled = false`).
3. Si el usuario pulsa **«Abortar lanzamiento»** antes de llegar a cero, la cuenta atrás se detendrá en seco y aparecerá un aviso de cancelación en pantalla.
4. Si la cuenta llega a **0**, el cohete despega con éxito, se detiene el intervalo y, **3 segundos después del despegue** (mediante `setTimeout`), el panel vuelve automáticamente a su estado original listo para una nueva misión.

🧱 Código Base Entregado (HTML y CSS de Práctica 1)

```html
<!-- 🚀 Estructura HTML de index.html entregada al alumno -->
<div class="panel">
    <h2>🚀 Control de Misión</h2>
    <div id="pantalla">10</div>
    <p id="mensaje">Sistemas preparados.</p>
    <button id="btnIniciar">Iniciar cuenta atrás</button>
    <button id="btnAbortar" disabled>Abortar lanzamiento</button>
</div>
```

🧭 Estados Funcionales del Panel

1. Inicial: Pantalla «10» | Mensaje «Sistemas preparados» | Iniciar habilitado | Abortar bloqueado.
2. Conteo: Pantalla descendente | Mensaje «Cuenta atrás en curso» | Iniciar bloqueado | Abortar activo.
3. Despegue: Pantalla «0» | Mensaje «Despegue realizado» ➔ `setTimeout` 3s ➔ Vuelta al Estado 1.

🧭 Guía de Desarrollo Paso a Paso (app.js)

1. **Paso 1 · Referencias al DOM:** Captura con `document.getElementById()`: `pantalla`, `mensaje`, `btnIniciar` y `btnAbortar`.
2. **Paso 2 · Variables mutables:** Declara con `let` en el ámbito superior: `segundos = 10;` y `idIntervalo = null;`.
3. **Paso 3 · Escuchador de arranque:** Al pulsar Iniciar, valida duplicados, actualiza textos, alterna `disabled` e inicia `setInterval` cada 1000 ms. Al llegar a 0, detén con `clearInterval` y dispara `setTimeout` de 3000 ms para restaurar.
4. **Paso 4 · Escuchador de abortado:** Al pulsar Abortar, frena con `clearInterval(idIntervalo)`, limpia el ID a `null`, muestra aviso y restaura botones.

💻 Solución Oficial Comentada (app.js)

```
// 🎯 PASO 1: Constantes de captura al DOM
const pantalla = document.getElementById("pantalla");
const mensaje = document.getElementById("mensaje");
const btnIniciar = document.getElementById("btnIniciar");
const btnAbortar = document.getElementById("btnAbortar");

// 🧠 PASO 2: Variables de estado en memoria
let segundos = 10;
let idIntervalo = null;

// 🚀 PASO 3: Escuchador del botón Iniciar
btnIniciar.addEventListener("click", () => {
    if (idIntervalo !== null) return; // 🛡️ Evitar duplicar intervalos

    segundos = 10;
    pantalla.textContent = segundos;
    mensaje.textContent = "⏳ Cuenta atrás en curso...";
    btnIniciar.disabled = true;
    btnAbortar.disabled = false;

    // ☄️ Iniciar cuenta atrás repetitiva cada 1000 ms
    idIntervalo = setInterval(() => {
        segundos--;
        pantalla.textContent = segundos;

        if (segundos === 0) {
            clearInterval(idIntervalo); // 🛑 Frenar el bucle
            idIntervalo = null;
            mensaje.textContent = "🔥 ¡Despegue realizado con éxito!";
            btnAbortar.disabled = true;

            // ⏳ Restauración diferida a los 3 segundos
            setTimeout(() => {
                pantalla.textContent = "10";
                mensaje.textContent = "Sistemas preparados.";
                btnIniciar.disabled = false;
            }, 3000);
        }
    }, 1000);
});

// 🛑 PASO 4: Escuchador del botón Abortar
btnAbortar.addEventListener("click", () => {
    if (idIntervalo !== null) {
        clearInterval(idIntervalo);
        idIntervalo = null;
        mensaje.textContent = "🛑 Lanzamiento abortado por el usuario.";
        btnAbortar.disabled = true;
        btnIniciar.disabled = false;
    }
});
```

🔋 Práctica 2: Simulador de Carga de Batería y Alerta Temporal

🎯 DATOS TÉCNICOS DE LA PRÁCTICA

Funciones: `setInterval()`, `clearInterval()`, `setTimeout()` y `clearTimeout()` · **Carpeta:** `practicas/practica_02_bateria/`

📖 Enunciado del Proyecto

Vas a programar el indicador interactivo de carga de una batería con las siguientes características técnicas:

1. Al pulsar **«Conectar cargador»**, la barra verde crece de **5% en 5% cada 200 milisegundos** hasta alcanzar el 100%. Se debe prevenir duplicar intervalos si ya está cargando o ya está completa.
2. En cualquier momento se puede pulsar **«Desconectar»** para pausar la carga en el porcentaje actual sin perder el progreso acumulado.
3. Al llegar al **100%**, se detiene el intervalo y aparece el aviso *«✅ ¡Batería completa al 100%!»*, el cual se auto-eliminará tras **4 segundos** usando `setTimeout()`.
4. Al pulsar **«Reiniciar batería»**, se restaura todo a 0% y, si había un temporizador de aviso de 4 segundos pendiente, debe cancelarse de inmediato mediante `clearTimeout()`.

🧱 Código Base Entregado (HTML de Práctica 2)

```html
<!-- 🔋 Estructura HTML de la tarjeta de batería -->
<div class="tarjeta">
    <h2>🔋 Estado de Carga</h2>
    <div class="carcasa-bateria">
        <div id="barraNivel">0%</div>
    </div>
    <p id="infoAviso">Cargador desconectado.</p>
    <button id="btnCargar">Conectar cargador</button>
    <button id="btnPausar">Desconectar</button>
    <button id="btnReiniciar">Reiniciar batería</button>
</div>
```

💡 El Doble Reseteo de Reiniciar

Un reinicio limpio debe apagar dos cosas: el intervalo de subida (`clearInterval(idCarga)`) y el aviso diferido si estaba en curso (`clearTimeout(idAvisoFinal)`). ¡Si olvidas cancelar el timeout, el mensaje se borrará de forma inesperada más tarde!

💻 Solución Oficial Comentada (app.js)

```
// 🎯 PASO 1: Constantes de referencia al DOM
const barraNivel = document.getElementById("barraNivel");
const infoAviso = document.getElementById("infoAviso");
const btnCargar = document.getElementById("btnCargar");
const btnPausar = document.getElementById("btnPausar");
const btnReiniciar = document.getElementById("btnReiniciar");

// 🎈 PASO 2: Variables de control de estado
let porcentaje = 0;
let idCarga = null;
let idAvisoFinal = null;

// ⚡ PASO 3: Escuchador para conectar el cargador
btnCargar.addEventListener("click", () => {
    if (idCarga !== null || porcentaje >= 100) return;
    infoAviso.textContent = "⚡ Cargando batería...";

    idCarga = setInterval(() => {
        porcentaje += 5;
        barraNivel.style.width = porcentaje + "%";
        barraNivel.textContent = porcentaje + "%";

        if (porcentaje >= 100) {
            clearInterval(idCarga);
            idCarga = null;
            infoAviso.textContent = "✅ ¡Batería completa al 100%!";

            // ⏳ Borrado diferido tras 4 segundos
            idAvisoFinal = setTimeout(() => {
                infoAviso.textContent = "Cargador desconectado.";
                idAvisoFinal = null;
            }, 4000);
        }
    }, 200);
});

// ⏸️ PASO 4: Escuchador para pausar la carga
btnPausar.addEventListener("click", () => {
    if (idCarga !== null) {
        clearInterval(idCarga);
        idCarga = null;
        infoAviso.textContent = "⏸️ Carga pausada.";
    }
});

// 🔄 PASO 5: Escuchador para reiniciar
btnReiniciar.addEventListener("click", () => {
    if (idCarga !== null) { clearInterval(idCarga); idCarga = null; }
    if (idAvisoFinal !== null) { clearTimeout(idAvisoFinal); idAvisoFinal = null; }
    porcentaje = 0;
    barraNivel.style.width = "0%";
    barraNivel.textContent = "0%";
    infoAviso.textContent = "Cargador desconectado.";
});
```

📊 Gran Tabla de Verificación y Autoevaluación

CRITERIO	PRÁCTICA 1 (COHETE ESPACIAL)	PRÁCTICA 2 (SIMULADOR DE BATERÍA)
Control de duplicados	`btnIniciar.disabled = true;` impide clics mientras corre.	`if (idCarga !== null || porcentaje >= 100) return;` al inicio.
Parada manual	`clearInterval(idIntervalo)` detiene la cuenta al abortar.	`clearInterval(idCarga)` congela el porcentaje al pausar.
Acción diferida única	`setTimeout( ... , 3000)` restaura a los 3s del despegue.	`setTimeout( ... , 4000)` borra el mensaje de batería llena.
Cancelación con clearTimeout	No se precisa (se autogestiona tras 3s).	`clearTimeout(idAvisoFinal)` aborta el borrado si se reinicia antes.